T.C.
TOKAT GAZİOSMANPAŞA ÜNİVERSİTESİ
BİLGİSAYAR TEKNOLOJİLERİ BÖLÜMÜ

MESLEKİ ÇÖZÜMLEME DERSİ
5. HAFTA LABORATUVAR RAPORU
(Veri Tabanı Uygulaması ve Geliştirme)

Proje / Senaryo Adı	Akilli Yoklama Sistemi
Öğrenci Adı Soyadı	ismail furkan alp
Öğrenci Numarası	254309042
GitHub Depo (Nihai Sürüm) Linki [ZORUNLU]	https://github.coSm/...

1. Projenin Amacı ve Haftalık Hedefler

Projenin amacı yüz tanıma teknolojisi kullanarak geleneksel yoklama sistemlerindeki zaman kaybını ve güvenlik açıklarını (başkasının yerine imza atma vb.) ortadan kaldıran, otonom ve güvenilir bir dijital yoklama kayıt sistemi geliştirmektir.

Bu haftaki hedefim 4. haftada tasarladığımız Varlık-İlişki (ER) diyagramını hiçbir yapısal değişikliğe uğratmadan SQLite üzerinde çalışan fiziksel bir veri tabanına dönüştürmektir.

2. Yapılan Çalışmaların Detaylı Açıklaması (ZORUNLU ALAN)

☐ ***1. KULLANICI Tablosunun Oluşturulması ve Veri Tipleri:***

- *Sistemin ana tablosudur. Her bir kullanıcının benzersiz bir kimliği olması gerektiği için Kullanici_ID sütunu INTEGER PRIMARY KEY AUTOINCREMENT (Otomatik artan birincil anahtar) olarak ayarlanmıştır.*
- *Ad, Soyad ve Rol (Öğrenci, Personel vb.) bilgileri metinsel veriler olduğu için TEXT veri tipi seçilmiştir. Bu sütunlara, boş bırakılamaması adına NOT NULL kısıtlaması eklenmiştir.*

☐ ***2. YÜZ_VEKTÖRÜ Tablosunun Oluşturulması ve BLOB Kullanımı:***

- *Bu tablo, kullanıcıya ait "1'e 1 (1:1)" ilişkiyle bağlanan yüz verisini tutar. Kimlik doğrulama, kişinin fotoğrafı üzerinden değil, yapay zekanın (OpenCV/dlib) fotoğraftan çıkardığı 128 boyutlu matematiksel matrisler üzerinden yapılacaktır.*

- ***Teknik Karar:*** *Matematiksel matris dizileri (array/numpy) düz metin (TEXT) olarak verimli saklanamaz. Bu nedenle Vektor_Verisi sütunu için, ikili verileri (binary) olduğu gibi saklayabilen* ***BLOB (Binary Large Object)*** *veri tipi tercih edilmiştir.*
- *Tablodaki Kullanici_ID sütunu FOREIGN KEY (Yabancı Anahtar) olarak KULLANICI tablosuna bağlanmış ve bu sayede veri tutarlılığı sağlanmıştır.*

3. YOKLAMA Tablosunun Oluşturulması ve 1:N İlişkisi:

- *Bu tablo, sistemin oluşturacağı giriş-çıkış zaman damgalarını tutar. Bir kişinin birden fazla yoklaması olabileceği için KULLANICI tablosuyla "1'e N (1:N)" ilişki kurulmuştur.*
- ***Teknik Karar:*** *Zaman kayıtlarında hata payını en aza indirmek için Tarih ve Saat bilgileri ayrı ayrı TEXT olarak tutulmuş (YYYY-MM-DD ve HH:MM formatında) ve yine Kullanici_ID FOREIGN KEY ile ana tabloya referans verilmiştir.*
- ***Referans Bütünlüğü (Cascade Kuralı):*** *Tüm Foreign Key bağlantılarına ON DELETE CASCADE kısıtlaması eklenmiştir. Böylece KULLANICI tablosundan bir kişi silindiğinde (örneğin mezun olduğunda veya işten çıktığında), ona ait yüz vektörleri ve geçmiş yoklama logları sistemde çöp veri olarak kalmadan otomatik olarak silinecektir.*

3. Veri Tabanı Şeması ve Tablo Yapıları

Tablo 1: KULLANICI

- ***Kullanici_ID:*** *INTEGER | Primary Key , AutoIncrement*
- ***Ad:*** *TEXT | Not Null (Boş Bırakılamaz)*
- ***Soyad:*** *TEXT | Not Null*
- ***Rol:*** *TEXT | Not Null*

Tablo 2: YÜZ_VEKTÖRÜ

- ***Vektor_ID:*** *INTEGER | Primary Key, AutoIncrement*
- ***Vektor_Verisi:*** *BLOB | Not Null (İkili Matris Verisi)*
- ***Kullanici_ID:*** *INTEGER | Foreign Key (FK) -> KULLANICI tablosundaki Kullanici_ID'ye bağlıdır. (Kısıtlama: ON DELETE CASCADE)*

Tablo 3: YOKLAMA

- ***Log_ID:*** *INTEGER | Primary Key , AutoIncrement*
- ***Tarih:*** *TEXT | Not Null (Format: YYYY-MM-DD)*

- ***Saat:*** *TEXT | Not Null (Format: HH:MM)*
- ***Kullanici_ID:*** *INTEGER | Foreign Key (FK) -> KULLANICI tablosundaki Kullanici_ID'ye bağlıdır. (Kısıtlama: ON DELETE CASCADE)*

4. Örnek SQL Sorguları / Kod Parçacıkları

Tabloları Oluşturma (CREATE TABLE) Sorguları:

1. Kullanıcı Tablosunu Oluşturma

```
CREATE TABLE KULLANICI (
   Kullanici_ID INTEGER PRIMARY KEY AUTOINCREMENT,
   Ad TEXT NOT NULL,
   Soyad TEXT NOT NULL,
   Rol TEXT NOT NULL
);
```

2***. Yüz Vektörü Tablosunu Oluşturma (KULLANICI'ya bağlı)***

```
CREATE TABLE YUZ_VEKTORU (
   Vektor_ID INTEGER PRIMARY KEY AUTOINCREMENT,
   Vektor_Verisi BLOB NOT NULL,
   Kullanici_ID INTEGER,
   FOREIGN KEY (Kullanici_ID) REFERENCES KULLANICI(Kullanici_ID) ON DELETE CASCADE
);
```

```
-- 3. Yoklama Tablosunu Oluşturma (KULLANICI'ya bağlı)
CREATE TABLE YOKLAMA (
   Log_ID INTEGER PRIMARY KEY AUTOINCREMENT,
   Tarih TEXT NOT NULL,
```

Saat TEXT NOT NULL,

Kullanici_ID INTEGER,

FOREIGN KEY (Kullanici_ID) REFERENCES KULLANICI(Kullanici_ID) ON DELETE CASCADE

);

Veri Ekleme (INSERT) Sorgusu Örneği:

INSERT INTO KULLANICI (Ad, Soyad, Rol)

VALUES ('İsmail Furkan', 'Alp', 'Öğrenci');

Veri Okuma ve Birleştirme (SELECT & JOIN) Sorgusu Örneği:

SELECT KULLANICI.Ad, KULLANICI.Soyad, KULLANICI.Rol, YOKLAMA.Tarih, YOKLAMA.Saat

FROM YOKLAMA

INNER JOIN KULLANICI ON YOKLAMA.Kullanici_ID = KULLANICI.Kullanici_ID;

5. Karşılaşılan Zorluklar ve Çözümler

SQLite'ta Tarih/Saat (DATETIME) Tipinin Olmaması:

- ***Zorluk:*** *YOKLAMA tablosunu oluştururken, giriş-çıkış zamanlarını kaydetmek için MySQL'de olan DATETIME veya TIME veri tiplerinin SQLite'ta yerleşik olarak bulunmadığını fark ettim. Bu durum zaman damgalarını nasıl tutacağım konusunda kafa karışıklığı yarattı.*
- ***Çözüm:*** *Araştırmalarım sonucunda SQLite'ın bu tür verileri TEXT olarak tutabildiğini öğrendim. İleride sorgularda sıralama (ORDER BY) işlemlerini kolaylaştırmak için, tarihleri standart "YYYY-MM-DD", saatleri ise "HH:MM" formatında TEXT olarak tutmaya karar verdim.*

